

PROJE 05 / 20 · ÇALIŞAN TEMEL SÜRÜM

URL Kısaltıcı ve Analitik

Güvenli URL doğrulamasıyla kısa link üret; yönlendirmeleri günlük say.

Gerçek yönlendirme · yerel analitik

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.

DYFULL-STACK LAB / 05 / 20Deniz DemoÇıkış

PORTFOLYO PROJESİ · ÇALIŞAN TEMEL SÜRÜM

URL Kısaltıcı ve Analitik

Güvenli URL doğrulamasıyla kısa link üret; yönlendirmeleri günlük say.

Gerçek yönlendirme · yerel analitik

Kısa bağlantı oluştur

Hedef URL

https://example.com

Son kullanım (opsiyonel)

mm/dd/yyyy, --:-- --

Oluştur

Bağlantılarım

KISA ADRES	HEDEF	TIKLAMA	İŞLEM
/r/Rn9eXn0v	https://example.com/docs	0	

Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

HTTP/HTTPS URL kontrolü · Rastgele base64url kısa kod · 302 ve no-store yönlendirme · Son kullanım ve kapatma · Günlük tıklama toplamı

Bu sürümde olmayanlar

Kod üretimi Base62 değil base64url kullanır. QR, konum, cihaz sınıfı, bot ayrımı ve benzersiz ziyaretçi metriği yok. Linkler yönlendirme içindir; hedef içerik sunucudan indirilmez.

DENİZ YARDIMCI · FULL-STACK LAB · 23.09.2026

1

02 / Backend ve veri modeli

Kısa kod kriptografik rastgele 6 bayttan üretilir. DB UNIQUE çakışmayı engeller ve kod yeniden denenir. Tıklama her başarılı yönlendirmede eklenir; ham IP saklanmaz. Cache-Control no-store tarayıcı önbelleğinin sayımı atlamasını azaltır. Süresi dolan veya kapalı bağlantı 410 döner.

Gerçek SQL şeması

```
CREATE TABLE links(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),code TEXT
UNIQUE,target TEXT,enabled INTEGER DEFAULT 1,expires TEXT);
```

```
CREATE TABLE clicks(id INTEGER PRIMARY KEY,link TEXT REFERENCES links(id),at TEXT);
```

```
CREATE INDEX clicks_by_link ON clicks(link,at);
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekiliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametrelili SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
POST /api/links	JSON: target, expires	201
GET /api/links	Gövde yok	200
PATCH /api/links/:id	JSON: enabled	200
GET /api/links/:id/stats	Gövde yok	200
GET /r/:code	Gövde yok	302 / 410

Somut istek örneği

```
POST /api/links
{"target":"https://example.com/docs","expires":null}
201 {"id":"<link-id>","code":"...", "shortPath":"/r/..."}
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- javascript: adresi reddedilmeli.
- Açılan link 302 ve doğru Location döndürmeli.
- Başarılı açılış sonrası sayaç artmalı.
- Kapalı link 410, yabancı kullanıcı istatistikte 404 almalı.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
await post('/links',{target:'javascript:alert(1)'},422);
const l=await post('/links',{target:'https://example.com/path?q=1'},201);
const redirect=await fetch(base+'/r/'+l.code,{redirect:'manual'});assert.equal(redirect.status,302);assert.equal(redirect.headers.get('location'),'https://example.com/path?q=1');
assert.equal((await get('/links'))[0].clicks,1);
await call('/api/links/'+l.id+'/stats','GET',undefined,404,other);
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

Link oluştur ve aç. Network panelinde 302 Location başlığını, panelde artan sayıyı kontrol et. Linki kapatıp 410 yanıtını gözle.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. QR üretimi ekle
- 2. Kötüye kullanım raporlama ekle
- 3. Redis cache geçersizleştirme tasarla
- 4. Tıklama yazımını kuyrukla ayır

Anlamli commit sırası

```
git add .
git commit -m "chore: add url-analytics baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

Yönlendirmeyi tarayıcı cache ederse tıklama metriği nasıl değişir? Rastgele kod ile hash arasındaki fark nedir?

Özel kurulum / ilerleme notu

Önce yerel sürümü tamamla. Sonraki aşamada PostgreSQL, migration, pagination ve gerçek kullanıcı yönetimini ihtiyaca göre ekle. Ortak kurulum ayrıntıları README.md ve başlangıç rehberindedir.